Guía de remediación de inyección SQL

Causa raíz, corrección del código y pruebas de regresión

Documento	SQL_INJECTION_REMEDIATION_GUIDE.pdf
Proyecto	sqli-vulnerable-lab (Flask + PyMySQL + MariaDB)
Versión del documento	1.0
Fecha	7 de noviembre de 2026
Elaborado por	Technical Writing + Security Engineering
Clasificación	Uso educativo interno - datos ficticios
Base del contenido	Código del repositorio: app/, database/, docker-compose.yml, Dockerfile, scripts/, tests/, docs/seguridad.md

Advertencia: Este laboratorio es intencionalmente vulnerable y esta destinado exclusivamente a la formación en entornos locales y controlados. No lo despliegues en redes publicas ni uses sus técnicas contra sistemas sin autorización expresa. Todos los datos son ficticios.

Índice

1. Análisis del código vulnerable . . . 3
2. Causa raíz . . . 3
3. Consultas parametrizadas . . . 4
3.1 Consultas afectadas y su corrección . . . 4
4. Validación de entradas . . . 4
5. Gestión de errores . . . 5
6. Buenas prácticas . . . 6
7. Comparación antes / después . . . 6
7.1 Resumen . . . 6
7.2 Login . . . 7
7.3 Buscador . . . 7
7.4 Producto . . . 7
8. Pruebas de regresión . . . 7
8.1 Pruebas existentes . . . 8
8.2 Pruebas sugeridas para la versión corregida . . . 8
8.3 Criterios de aceptación . . . 9
9. Plan de remediación . . . 9
10. Referencias . . . 9

1. Análisis del código vulnerable

Esta sección analiza el código real del repositorio. El laboratorio mantiene a proposito dos formas de ejecutar consultas en *app/db.py*: una cruda y otra parametrizada. Las rutas vulnerables usan la primera.

python

```
# app/db.py
def query_raw(sql):
"""Ejecuta SQL concatenado tal cual. VULNERABLE a inyeccion SQL."""
conn = get_connection()
try:
with conn.cursor() as cursor:
cursor.execute(sql)
return cursor.fetchall()
finally:
conn.close()
def query_parametrized(sql, params):
"""Ejecuta SQL con marcadores %s y parametros. Via segura."""
conn = get_connection()
try:
with conn.cursor() as cursor:
cursor.execute(sql, params)
return cursor.fetchall()
finally:
conn.close()
```

Archivo	Función / ruta	Problema
app/routes/auth.py	login() - /login	f-string con username y password; query_raw().
app/routes/auth.py	secure_login() - /secure-login	Correcto: %s y query_parametrized().
app/routes/scenarios.py	search() - /search	f-string con q dentro de LIKE; error expuesto con str(exc).
app/routes/scenarios.py	product() - /product	f-string con id; error expuesto con str(exc).
app/routes/dashboard.py	index() - /dashboard	SQL estatica sin entrada de usuario (sin riesgo de inyección).
app/db.py	query_raw()	Herramienta didáctica: ejecuta SQL ya concatenado.

La única función que recibe parámetros de forma segura es *query_parametrized()*. Las rutas vulnerables la ignoran: construyen la sentencia completa en Python y la pasan a *query_raw()*.

2. Causa raíz

La causa raíz común a los cuatro escenarios es la **perdida de la frontera entre datos e instrucciones**: la entrada del usuario se interpola en la cadena SQL antes de llegar al driver. Los factores que la agravan son la ausencia de validación y la divulgación de errores.

Factor	Donde aparece	Efecto
Concatenación con f-strings	auth.py (login), scenarios.py (search, product)	La entrada altera la estructura de la sentencia.
Sin validación de tipo o formato	Todos los parámetros se leen con request.args/form.get	Cualquier cadena, incluidas comillas y comentarios, llega al motor.
Divulgación de errores	scenarios.py: error = str(exc)	El atacante recibe el mensaje SQL real y la versión del motor.
Contraseñas en texto plano	seed.sql y comparación en la consulta	La contraseña se compara en claro dentro del SQL.
Exposición del SQL ejecutado	login.html, results.html	La respuesta muestra la consulta final, facilitando el ajuste del payload.

Información: No existe una única "solución mágica": la parametrización corrige la causa raíz, mientras que la validación y la gestión de errores reducen el impacto si aparece otra vulnerabilidad.

3. Consultas parametrizadas

Una consulta parametrizada envía la sentencia y los valores por separado. El motor analiza la estructura una sola vez y trata los parámetros como literales, por lo que una comilla o un comentario en el valor no pueden cambiar la semantica.

python

```
# VULNERABLE: la entrada forma parte de la sentencia
sql = f"SELECT id, name FROM products WHERE id = '{pid}'"
query_raw(sql)
# SEGURO: la sentencia es fija y el valor viaja aparte
sql = "SELECT id, name FROM products WHERE id = %s"
query_parametrized(sql, (pid,))
```

En PyMySQL el marcador es *%s* y la tupla de parámetros se pasa como segundo argumento de *cursor.execute()*. Es importante no confundir el marcador con un formato de cadenas: el driver escapa el valor, no lo interpola.

3.1 Consultas afectadas y su corrección

Ruta	Consulta actual	Consulta corregida (propuesta)
/login	SELECT id, username, role FROM users WHERE username = '{u}' AND password = '{p}'	SELECT id, username, role FROM users WHERE username = %s AND password = %s
/search	SELECT id, name, price, stock FROM products WHERE name LIKE '%{q}%'	SELECT id, name, price, stock FROM products WHERE name LIKE %s (el patrón se calcula en Python: '%' || q || '%')
/product	SELECT id, name, price, stock FROM products WHERE id = '{pid}'	SELECT id, name, price, stock FROM products WHERE id = %s

Para el caso del LIKE, el patrón con comodines se construye en Python y se pasa como un único parámetro:

python

```
patron = '%' + q[:80] + '%'
sql = "SELECT id, name, price, stock FROM products WHERE name LIKE %s"
rows = query_parametrized(sql, (patron,))
```

4. Validación de entradas

La validación es una capa complementaria: acota el formato, la longitud y el rango de las entradas antes de usarlas. No sustituye a la parametrización, pero reduce la superficie y mejora los mensajes de error.

Parámetro	Regla propuesta	Código de ejemplo
id (/product)	Entero positivo dentro de un rango razonable.	valor = int(raw) # con control de errores
q (/search)	Cadena no vacia, máximo 80 caracteres, sin controlar la semantica SQL.	q = request.args.get('q', ''.strip()[:80]
username (/login)	Longitud 3-50, caracteres permitidos [A-Za-z0-9_.@-].	re.fullmatch(r"[A-Za-z0-9_.@-]{3,50}", u)
password (/login)	Longitud mínima; nunca se registra ni se muestra.	if len(p) < 8: error

python

```
def _validar_id(raw):
"""Convierte el parametro id a entero y acota el rango."""
try:
valor = int(raw)
except (TypeError, ValueError):
abort(400, "El identificador debe ser un numero entero.")
if valor < 1 or valor > 10000000:
abort(400, "Identificador fuera de rango.")
return valor
# Uso en la ruta
pid = _validar_id(request.args.get("id", ""))
sql = "SELECT id, name, price, stock FROM products WHERE id = %s"
rows = query_parametrized(sql, (pid,))
```

Regla práctica

Válida lo que la aplicación espera, no lo que "parece peligroso": una lista blanca de formatos es más robusta que una lista negra de payloads.

5. Gestión de errores

El código actual captura la excepción y la convierte en texto para la plantilla (*error = str(exc)*). En una aplicación real eso revela el motor, la versión y la estructura de la consulta. La corrección consiste en registrar el detalle en el servidor y mostrar un mensaje generico al usuario.

python

```
# VULNERABLE (codigo actual)
except Exception as exc:
error = str(exc)
return render_template("results.html", ..., error=error)
# PROPUESTA
import logging
log = logging.getLogger(__name__)
except Exception:
log.exception("Error de base de datos en la ruta /search")
error = "No se pudo completar la busqueda. Intentalo de nuevo."
return render_template("results.html", ..., error=error)
```

Con esta corrección, los escenarios dejan de mostrar el error SQL (que es el objetivo didáctico del laboratorio), por lo que en el entorno de formación debe conservarse una versión del laboratorio sin remediar para los ejercicios y aplicar la corrección en una copia.

Advertencia: Para el laboratorio educativo, la versión con errores visibles debe mantenerse separada (por ejemplo, una rama o un contenedor etiquetado como "vulnerable") de la versión corregida que sirva de referencia.

6. Buenas prácticas

Capas complementarias: ninguna sustituye a las demas

Práctica	Descripción	Prioridad
Consultas parametrizadas	Única defensa completa contra la inyección SQL.	Obligatoria
Validación de entradas	Tipo, longitud, formato y rango; lista blanca.	Alta
Gestión de errores	Registro interno y mensajes genericos al usuario.	Alta
Hash de contraseñas	bcrypt o argon2; nunca texto plano.	Alta
Menor privilegio en BD	La cuenta de aplicación solo necesita SELECT/INSERT/UPDATE sobre lo necesario.	Alta
Principio de defensa en profundidad	Capas complementarias: WAF, limites de tasa, revisión de código.	Media
Gestión de secretos	SECRET_KEY aleatorio por despliegue; nunca en el repositorio.	Media
Cabeceras y cookies seguras	HSTS, CSP, X-Content-Type-Options, Secure, SameSite.	Media
Pruebas de regresión	Verificar que los payloads conocidos ya no funcionan.	Obligatoria

Adicionalmente, el proyecto ya aplica algunas medidas estructurales: contenedores sin privilegios elevados, base de datos sin puertos publicados, enlace a 127.0.0.1 y ausencia de directorios sensibles montados.

7. Comparación antes / después

7.1 Resumen

Aspecto	Antes	Después
Login	f-string + query_raw()	%s + query_parametrized()
Buscador	LIKE con f-string	LIKE con %s y patrón calculado
Producto	id interpolado como texto	id validado como entero y %s
Errores	str(exc) en la plantilla	logging + mensaje generico
Contraseñas	texto plano comparado en SQL	hash (bcrypt/argon2) y comparación con el hash
Validación	ninguna	tipo, longitud y rango acotados

7.2 Login

```python
# ANTES (app/routes/auth.py) - VULNERABLE
sql = (
"SELECT id, username, role FROM users "
f"WHERE username = '{username}' AND password = '{password}'"
)
rows = query_raw(sql)
# DESPUES (propuesta)
sql = "SELECT id, username, role, password FROM users WHERE username = %s"
rows = query_parametrized(sql, (username,))
if rows and check_password_hash(rows[0]["password"], password):
# credenciales validas
```

7.3 Buscador

```python
# ANTES (app/routes/scenarios.py) - VULNERABLE
sql = f"SELECT id, name, price, stock FROM products WHERE name LIKE '%{q}%'"
# DESPUES (propuesta)
q = request.args.get("q", "").strip()[:80]
if not q:
return render_template("results.html", results=[], sql=None, error=None)
patron = "%" + q + "%"
sql = "SELECT id, name, price, stock FROM products WHERE name LIKE %s"
rows = query_parametrized(sql, (patron,))
```

7.4 Producto

```python
# ANTES (app/routes/scenarios.py) - VULNERABLE
sql = f"SELECT id, name, price, stock FROM products WHERE id = '{pid}'"
# DESPUES (propuesta)
pid = _validar_id(request.args.get("id", ""))
sql = "SELECT id, name, price, stock FROM products WHERE id = %s"
rows = query_parametrized(sql, (pid,))
```

Nota

Los bloques "después" son propuestas ilustrativas coherentes con la arquitectura del proyecto; no reemplazan la revisión humana del código.

8. Pruebas de regresión

Las pruebas de regresión verifican que la corrección no rompe la funcionalidad y que los vectores conocidos dejan de funcionar. El repositorio ya incluye una suite pensada para el laboratorio vulnerable; sobre la versión corregida deben añadirse casos específicos de bloqueo.

8.1 Pruebas existentes

Archivo	Prueba	Que verifica
test_login_vulnerable.py	test_login_normal_redirige_a_dashboard	Credenciales válidas -> 302 a /dashboard.
test_login_vulnerable.py	test_login_bypass_clasico	El bypass autentica en /login (302).
test_login_secure.py	test_secure_login_normal	Credenciales válidas -> 302 a /dashboard.
test_login_secure.py	test_secure_login_bypass_bloqueado	El bypass NO autentica: 200 con error y /dashboard -> 302 a /login.
test_sqli_scenarios.py	test_search_error_based	El error SQL es visible en la respuesta.
test_sqli_scenarios.py	test_product_boolean_based	Condición verdadera y falsa producen respuestas distintas.
test_sqli_scenarios.py	test_product_time_based	SLEEP(5) produce un retardo mayor de 4 s.
test_docker.py	test_contenedores_del_lab_activos	sqli-lab-web y sqli-lab-db estan activos.
test_docker.py	test_db_no_expone_puertos	La base de datos no publica puertos.
test_docker.py	test_web_expone_solo_127_0_0_1_8080	El web solo escucha en 127.0.0.1:8080.

bash

```
# Requisitos: laboratorio en marcha y venv con pytest + requests
source venv/bin/activate
pytest -v
# URL alternativa (opcional)
LAB_URL=http://127.0.0.1:8080 pytest -v
```

8.2 Pruebas sugeridas para la versión corregida

Caso	Entrada	Resultado esperado
Login legitimo	admin / Password123!	302 a /dashboard; sesión creada.
Login con payload	admin' OR '1'='1' -- -	200 con "Credenciales inválidas"; sin sesión.
Buscador con comilla	q='	200 con 0 resultados o mensaje controlado; sin error SQL.
Buscador legitimo	q=Laptop	1 resultado.
ID no numerico	id=abc	400 con mensaje de validación.
ID con payload	id=1' AND '1'='2	400 (validación) o 0 resultados; nunca una consulta alterada.
ID válido	id=1	1 resultado.

Ejemplo de prueba de regresión (propuesta):

python

```
def test_login_parametrizado_bloquea_bypass(base_url):
response = requests.post(
f"{base_url}/login",
data={"username": "admin' OR '1'='1' -- -", "password": "loquesea"},
allow_redirects=False,
timeout=15,
)
assert response.status_code == 200
assert "Credenciales invalidas" in response.text
dashboard = requests.get(f"{base_url}/dashboard", allow_redirects=False,
timeout=15)
assert dashboard.status_code == 302
assert "/login" in dashboard.headers.get("Location", "")
```

8.3 Criterios de aceptación

- Todas las consultas con entrada de usuario usan marcadores %s.
- Ninguna respuesta contiene errores SQL, consultas ejecutadas ni trazas de excepciones.
- Los payloads de la tabla de la sección 8.2 no producen autenticación ni alteración de resultados.
- La funcionalidad legitima (login, busqueda, producto, dashboard) sigue operativa.
- La suite completa de pytest termina sin errores.
- Las pruebas de infraestructura confirman que la base de datos no publica puertos y que el web solo escucha en 127.0.0.1.

9. Plan de remediación

Fase	Actividad	Esfuerzo estimado	Riesgo de regresión
Fase 0	Inventario de consultas y parámetros.	0,5 h	Nulo
Fase 1	Parametrizar /login, /search y /product.	2-4 h	Bajo
Fase 2	Anadir validación de entradas y limites de longitud.	2 h	Bajo
Fase 3	Gestión de errores: logging y mensajes genericos.	1-2 h	Bajo
Fase 4	Hash de contraseñas y migración del seed.	2-3 h	Medio
Fase 5	Cabeceras de seguridad y atributos de cookie.	1 h	Bajo
Fase 6	Pruebas de regresión y documentación.	2 h	Nulo

Las fases 1 a 3 son las que corrigen la inyección SQL y pueden ejecutarse juntas. La fase 4 es independiente y solo afecta a la autenticación. Se recomienda mantener el laboratorio vulnerable original intacto como material didáctico y aplicar la remediación sobre una copia.

10. Referencias

Referencia	Utilidad
OWASP SQL Injection Prevention Cheat Sheet	Patrones de parametrización segun motor y lenguaje.
OWASP Top 10 2021 - A03:2021 Injection	Contexto y clasificación del riesgo.
CWE-89 (SQL Injection)	Definición formal de la debilidad.
docs/seguridad.md del repositorio	Reglas de uso del laboratorio.
tests/ del repositorio	Suite de pruebas de los escenarios y de infraestructura.

Cierre

La remediación completa requiere parametrización, validación y gestión de errores aplicadas de forma consistente, verificadas con las pruebas de regresión descritas en la sección 8.